

From Intent to Execution: Constraining Agent Tool Calls in Dynamic Geometric Spaces（从意图到执行：在动态几何空间中约束智能体工具调用）

这份文档给主图绘制前阅读使用。内容只讲 CLAFR（Confidence-Lifted Action Feasible Region，置信提升的动作可行域）的方法论、算法流程、关键公式和工程边界，绘制者理解方法后自行决定视觉呈现。

1. 方法论介绍

CLAFR 关注的是工具型智能体中最关键的一步：模型已经提出了一个具体工具调用，系统是否允许它真正作用到外部环境。传统叙述容易把安全问题放在输入文本、系统提示或攻击检测上；CLAFR 把问题放到执行边界上，直接检查“这个工具调用在当前证据下能不能执行”。

CLAFR 的基本视角是：每个工具调用都可以看成高维动作与证据空间中的一个点。这个点同时包含工具名、参数、用户意图、参数来源、可信授权、状态依据、对象是否明确、写入是否必要、是否涉及外部目的地，以及隐私、金融、不可逆副作用和不可信控制信号等信息。这样，一个看起来语法正确的工具调用会被拆成多个可检查的执行条件。

在这个高维空间中，CLAFR 不用单一安全分数决定执行。它根据当前用户任务、策略和运行时证据构造一个动态可行域。可行域由多种边界共同限定：有些边界像半空间一样约束单项条件，例如模式完整性、可信任务支撑和关键参数落地；有些边界刻画多种风险共同作用后的联合形状，例如隐私外发、金融暴露、状态写入和不可信控制之间的组合风险。动作点落在这些边界共同允许的区域内，调用才进入工具执行器；动作点越过任一硬边界，系统阻断或要求澄清。

这个方法论的核心贡献是把“从意图到执行”写成一个动态几何约束过程。可信事实可以支撑参数和状态，可信确认可以扩大可执行范围，不可信观察只能增加风险或触发更严格的约束。随着状态读取、用户确认和工具反馈不断进入上下文，同一个工具调用对应的动作点会移动，当前可行域也会收缩或扩张。

2. 算法对象

CLAFR 的算法对象聚焦于一个待执行工具调用，整段对话用于提供可信意图、执行策略和运行时证据。令可信用户意图为 u ，可信执行策略为 p ，第 t 步工具执行前的证据状态为 e_t 。规划器提出的具体工具调用写作：

$$a_t = (f_t, \rho_t),$$

其中 f_t 是工具， ρ_t 是参数映射。一次决策的输入是 u, p, e_t, a_t ，输出是允许执行、阻断或要求澄清。抽象地看，CLAFR 计算的是：

$$D_t(u, p, e_t, a_t) \in \{\text{Allow}, \text{Block}\}.$$

如果动作被允许，执行器原样提交 a_t ，工具返回新的观察 o_{t+1} 。这个观察会更新下一步证据状态：

$$e_{t+1} = T(e_t, a_t, o_{t+1}).$$

这条更新式表达了 CLAFR 的动态性：新可信证据可能补足先前缺失的条件，新不可信影响也可能激活或违反风险约束。

3. 动作与证据提升

CLAFR 的第一步是把具体工具调用提升为高维动作与证据点。编码器把用户意图、策略、证据状态和动作本身联合编码为：

$$z_t = \phi(u, p, e_t, a_t) \in [0, 1]^d.$$

这里 d 表示当前实现采用的坐标维度。旧论文草稿中用 $d = 28$ 说明方法形态；当前代码中维度以 `FEATURE_NAMES` 为准，已经随实现扩展。主图和方法说明不需要锁死具体维度，只需要表达“高维动作与证据空间”。

为了让读者理解各维度的作用，可以把 z_t 概念上分成四组：

$$z_t = [g_t, s_t, q_t, r_t].$$

其中 g_t 表示意图与工具对齐， s_t 表示可信执行支撑， q_t 表示状态落地与证据质量， r_t 表示效果相关风险。对应到自然语言，坐标会覆盖任务支撑、工具适配、参数可信来源、关键参数落地、状态读取必要性、对象唯一性、写入授权、可信确认、模式有效性、置信度、不确定性、不可信控制信号、外部目的地、隐私外发、金融风险、写入风险和不可逆风险。

高维表示的目的，是保留执行条件之间的差异。一个动作可能和用户任务相关，但缺少授权；也可能参数来自可信状态，但目的地来自不可信文本；还可能每个风险单独看都不强，组合后形成不可接受的外

部副作用。把这些因素放在不同坐标上，后续几何边界才能分别约束单项条件和联合风险。

4. 动态几何可行域

给定 p 和 e_t ，CLAFR 会编译当前时刻的可行域。可行域由两类约束共同形成。

第一类是半空间式语义边界，用来表达必须满足的硬条件。对第 j 个语义边界，可以写成：

$$w_{t,j}^\top z \leq b_{t,j}, \quad j \in \mathcal{H}_t.$$

这里 $\mathcal{H}_t$ 表示当前激活的语义边界集合。它们约束工具模式完整性、可信任务支撑、状态清洁、实体落地、关键副作用参数落地，以及可信方向压过不可信控制方向。

第二类是联合风险边界，用来处理多种风险与可信支撑之间的耦合关系。令 $R_k z$ 选取风险坐标， $C_k z$ 选取可信支撑坐标，第 k 个联合风险约束可写成：

$$q_{t,k}(z) = \|W_{t,k} R_k z\|_2 - v_{t,k}^\top C_k z \leq \beta_{t,k}, \quad k \in \mathcal{K}_t.$$

这里 $\mathcal{K}_t$ 表示当前激活的联合风险边界集合。前半项把隐私外发、外部目的地、写入、金融、不可逆影响和不可信控制等风险聚合起来；后半项表示可信授权、可信确认、状态落地、关键参数落地和必要读取带来的支撑。这个形式让多个弱风险可以共同放大，也限制了“高相似度”对缺授权、缺落地或高风险副作用的过度补偿。

因此，当前动作可行域可以写成：

$$\mathcal{F}_t = [0, 1]^d \cap \bigcap_{j \in \mathcal{H}_t} \{z : w_{t,j}^\top z \leq b_{t,j}\} \cap \bigcap_{k \in \mathcal{K}_t} \{z : q_{t,k}(z) \leq \beta_{t,k}\}.$$

这条式子是 CLAFR 最重要的几何表达：动作空间先被限制在高维单位盒内，再被多个半空间边界和联合风险边界共同切分。可行域由当前证据和策略即时编译，是一个随任务、状态和风险共同变化的高维区域。

5. 边界差距与执行决策

CLAFR 在判断动作是否越界的同时，记录它离每个边界有多远。对半空间式语义边界，边界差距定义为：

$$m_{t,j}^{\text{lin}}(z) = b_{t,j} - w_{t,j}^\top z, \quad \bar{m}_{t,j}^{\text{lin}}(z) = \frac{m_{t,j}^{\text{lin}}(z)}{\max(\|w_{t,j}\|_2, \epsilon)}.$$

对联合风险边界，边界差距定义为：

$$m_{t,k}^{\text{joint}}(z) = \beta_{t,k} + v_{t,k}^\top C_k z - \|W_{t,k} R_k z\|_2.$$

令 $\mathcal{G}_t$ 表示所有硬约束， $\bar{m}_{t,g}$ 表示归一化后的边界差距。动作的执行余量定义为所有硬约束中最小的边界差距：

$$M_t(a_t) = \min_{g \in \mathcal{G}_t} \bar{m}_{t,g}(z_t).$$

最终决策可以写成：

$$D_t(u, p, e_t, a_t) = \begin{cases} \text{Allow}, & M_t(a_t) \geq 0, \\ \text{Block}, & M_t(a_t) < 0. \end{cases}$$

这组公式说明了 CLAFR 的执行逻辑：所有硬边界都通过，动作进入工具执行器；任一硬边界被穿过，动作被阻断或要求澄清，并返回违反的约束、越界程度和修复提示。当前 CLAFR 主路径不会自动改写动作，也不会生成替代工具调用。

6. 算法流程

CLAFR 的算法流程可以分成七步。这七步是论文方法的主线，需要和具体工程文件、基准适配器、日志格式分开理解。

第一步，收集当前执行边界的信息。系统读取用户任务、策略、工具模式、已知可信状态、可信授权、可信确认和不可信观察，并组织成证据状态 e_t 。这里的关键是区分“谁能给授权”和“谁只能给事实”。用户和系统可信通道可以给出目标与授权；工具输出可以补充事实和状态；不可信内容只进入风险侧。

第二步，形成候选工具调用。规划器给出一个或多个可能执行的工具调用。CLAFR 接管的是执行前最后一层判断，候选动作可以来自普通 agent planner，也可以来自任意工具调用生成器。

第三步，计算动作点 $z_t = \phi(u, p, e_t, a_t)$ 。这一步把工具名、参数、可信任务、状态证据和不可信观察放入同一个高维动作与证据空间。

第四步，编译动态可行域 $\mathcal{F}_t$ 。策略和证据决定哪些半空间式边界、哪些联合风险边界被激活。当存在不可信内容时，额外激活可信方向与不可信方向之间的约束；当任务涉及隐私、金融或写入副作用时，激活对应的联合风险边界。

第五步，计算动作点相对每个边界的差距。正值表示动作仍在该边界允许的一侧，负值表示动作已经越界。软约束可以影响排序或提示，硬约束决定动作是否可行。

第六步，做执行决策。如果至少有一个候选动作位于可行域内，CLAFR 在可行动作中选择最终执行的调用。选择时会综合任务效用、动作位于可行域内部的程度、置信度和候选之间的接近程度。如果所有候选动作都越过硬边界，CLAFR 返回阻断或澄清。

第七步，更新下一步证据状态。允许执行的工具调用会产生新的观察 o_{t+1} ，并通过 $e_{t+1} = T(e_t, a_t, o_{t+1})$ 更新证据。新的可信事实可能补足对象、收件人、金额、时间或当前状态；新的不可信重定向可能提高风险，使之后的动作更难进入可行域。

7. 几何空间如何体现贡献

CLAFR 的空间是动作与证据共同定义的执行空间，每个坐标都服务于一个安全或可执行性条件。一个工具调用在这个空间中的位置，反映它当前受到多少可信任务支撑、参数是否来自可信证据、执行对象是否明确、是否有授权、是否会造成外部副作用，以及不可信内容是否在推动动作方向。

多种形状共同约束是 CLAFR 的关键。半空间式边界适合表达硬条件，例如模式必须完整、关键参数必须落地、外部副作用必须有可信任务支撑。联合风险边界适合表达组合风险，例如单独看外部目的地、敏感内容、写入效果和不可信文本对齐都只是弱信号，但它们叠加后形成不可接受的风险。不同边界的交集让 CLAFR 同时表达“缺一个硬条件就不能执行”和“多个弱风险会共同放大”。

动态性来自证据与边界的共同变化。可信状态读取会改变动作点的参数支撑，用户确认会改变授权坐标，不可信观察会改变风险坐标；策略和上下文还会改变当前激活的约束集合。这样，CLAFR 可以表达同一个工具调用在不同证据状态下的不同命运：缺少对象时阻断，读到可信对象后允许；缺少授权时阻断，得到可信确认后允许；出现不可信重定向后重新收紧。

旧论文草稿还给出了三个有助于理解的性质。第一，固定证据和激活约束后， $\mathcal{F}_t$ 是凸的，因为它由单位盒、半空间和联合风险约束的凸子水平集相交而成。第二，联合风险边界满足风险-支撑单调性：风险坐标增加且可信支撑不增加时，边界差距不会变好；风险固定且可信支撑增加时，边界差距不会变差。第三，在编码器、编译器和证据快照固定时，保存 z_t 、 $\mathcal{F}_t$ 和所有硬约束差距就足以复现允许或阻断决策。

8. 算法与工程实现的分界

论文方法应该讲算法层：运行时证据如何组织，工具调用如何提升为高维动作与证据点，动态可行域如何由多种边界形成，动作如何根据硬约束被允许、阻断或澄清。

工程实现负责把这个算法落地。当前代码中，`RuntimeEvidence` 和 `ToolAction` 对应算法输入，`ConfidenceLiftedEncoder` 对应动作与证据提升，`PolicyCompiler` 对应可行域编译，`LinearFacet` 与 `RiskBudgetCone` 对应两类几何边界，`ConfidenceLiftedActionSelector` 对应最终选择与阻断逻辑，`ActionCertificate` 对应可追踪的决策证书。

格式投影、观察投影、回复投影、AgentDojo 适配器、日志字段和实验消融脚本属于工程闭环。它们有助于完整运行和评估 CLAFR，方法论主体仍然是执行边界上的高维动作与证据表示，以及由多种几何边界共同形成的动态可行域。